

Abilium: Qualitätssicherung & Testing

Sascha Friedli

sascha.friedli@students.unibe.ch

Testkonzepte

- Automatisierte Tests mit dem Odoo-Testframework
- Manuelle Tests für Useability

Odoo Testframework

- Automatisierte Tests innerhalb Odoo-Umgebung
- Test-Basisklassen mit Odoo-spezifischen Testfunktionen
- Tests sind integriert in Projektstruktur
- Durch ORM arbeiten Testfälle mit echter Testdatenbank

```
PSE_TRMNL_ODOO_CONNECTOR/  
└─ addons/  
    └─ trmnl/  
        ├── controllers/  
        ├── models/  
        ├── security/  
        ├── views/  
        ├── utils/  
        ├── tests/  
        │   ├── test_api_common.py  
        │   ├── test_api_display.py  
        │   ├── test_api_log.py  
        │   ├── test_api_setup.py  
        │   └─ test_device_refresh_rate.py  
        └─ __init__.py
```

Odoo Testframework

HTTP-Case für API-Integrationstests

- 3 api-calls: api/setup, api/display, api/log
- Setup registriert ein neues Gerät korrekt und gibt nur die erwarteten Felder zurück.

```
self.assertEqual(
    setup_payload,
    {
        "status": 200,
        "api_key": api_token,
        "friendly_id": registered_device.friendly_id,
        "image_url": registered_device.image_url,
    },
)
```

Odoo Testframework

HTTP-Case für API-Integrationstests

- 3 api-calls: api/setup, api/display, api/log
- Geräte erhalten abhängig von ihrem Zustand die korrekte Display-Antwort

```
self.assertEqual(
    display_payload,
    {
        "status": 0,
        "image_url": refreshed_device.image_url,
        "filename": "abilium_test_screen",
        "refresh_rate": 1800,
        "special_function": "none",
        "action": "",
    },
)

self.assertEqual(refreshed_device.display_request_count, 1)
```

```
self.assertEqual(display_payload, {"status": 202})
self.assertEqual(stub_device.approval_state, APPROVAL_STATE_UNKNOWN_DEVICE)
```

Odoo Testframework

HTTP-Case für API-Integrationstests

- 3 api-calls: api/setup, api/display, api/log
- Log-Requests speichern gültige Logeinträge und lehnen ungültige Geräte oder Tokens mit 401 ab.

```
def test_api_log_invalid_token_returns_401_without_body(self):
    setup_context = self._register_device_through_setup()
    registered_device = setup_context["device"]

    log_response = self._call_json_endpoint(
        "/api/log",
        headers=self._log_headers(self.BAD_TOKEN, registered_device.mac_address),
        payload=self._log_payload(),
    )

    self.assertEqual(self._response_status(log_response), 401)
    self.assertEqual(self._response_text(log_response), "")
```

```
self.assertEqual(self._response_status(log_response), 204)
self.assertEqual(len(log_entries), 2)
self.assertEqual(refreshed_device.log_entry_count, 2)
```

Odoo Testframework

Transaction-Case für Model/Unit Tests

- Test von Grenzwerten und Validierungsfehler

```
def test_boundary_below_lower_is_rejected_on_create(self):  
    """Creating a device below the minimum rate should raise ValidationError."""  
    with self.assertRaises(ValidationError):  
        self._create_device(REFRESH_RATE_MIN_SECONDS - 1)
```

Manuelles Testen

- Darstellung Inhalt auf TRMNL Display
 - Test Usability Odoo Interface zur Konfiguration
- Fokus auf Benutzererfahrung

